



۸۱۰۱۰۱۴۹۲

شبکه‌های کامپیوتری
نام و نام خانوادگی: علیرضا کریمی



نوت ۲۲ مهر

فصل دوم: لایه *Application* اسلاید ۱ تا ۳۲ [۱]

بعضی کاربردهای شبکه‌ای معروف

شبکه‌های اجتماعی، وب، پیام‌رسان‌ها، ایمیل، بازی‌های شبکه‌ای چند نفره، اپلیکیشن‌های *video streaming* (مثل یوتیوب)، اشتراک فایل به صورت هم‌تا به هم‌تا *P2P*، انتقال صدا از طریق *IP*، ویدئو کنفرانس‌های *real – time*، جستجوهای اینترنتی و لاگین از راه دور (*remote login*).

ساخت اپلیکیشن‌های شبکه

باید اپلیکیشن‌ها به گونه‌ای ساخته شوند که در *end user*های مختلف (پلتفرم‌های مختلف) قابل اجرا باشند. اصلاً لازم نیست اپلیکیشن‌ها را به صورت *compatible* با هسته شبکه برنامه‌نویسی کنیم، چون اصلاً هسته شبکه، برنامه‌ها را ران نمی‌کند.

اپلیکیشن‌ها از طریق شبکه با هم ارتباط برقرار می‌کنند، مثلاً اپ‌های تحت وب، با *browser* ارتباط می‌گیرند.

پارادایم (الگو یا معماری) *client – server*

از پروتکل‌های این معماری می‌توان به *HTTP, IMAP, FTP* و غیره اشاره کرد. در این نوع معماری، سرورها همیشه روشن هستند. آدرس *IP* آنها به صورت دائمی ثابت است. و برا اینکه بتوان آنها را اسکیل کرد، در *data center*ها قرار می‌گیرند. در سمت *client*ها آنها گاهی *intermittently* با سرور تماس برقرار کرده و به آن متصل می‌شوند. می‌تواند آدرس *IP* آنها متغیر باشد. همچنین کلاینت‌ها به طور مستقیم با هم ارتباط نمی‌گیرند.

پارادایم (الگو یا معماری) *peer – to – peer*

این سیستم، خود مقیاس پذیر است. در واقع هر *peer* سرویس جدیدی را به شبکه اضافه می‌کند و یا درخواست سرویس می‌کند.

اینگونه نیست که سروری باشد که همیشه روشن است. سیستم‌های پایانی می‌توانند برای دریافت یا ارسال سرویس به طور مستقیم با هم ارتباط بگیرند. آنها گاهی و به صورت *intermittent* به اینترنت متصل می‌شوند و همچنین *IP* آنها می‌تواند متغیر باشد.

به همین دلایل، معماری آنها می‌تواند پیچیده باشد.

راه‌های *file sharing* به صورت *P2P*، مثالی از کاربرد این نوع پارادایم است.

process

درواقع، *process* یک برنامه کوچکی است که در یک سیستم اجرا می‌شود. هر سیستم می‌تواند شامل چند پروسس باشد. پروسس‌های یک سیستم از طریق روش‌های *inter-process communication* (که توسط سیستم‌عامل مشخص شدند) با هم ارتباط برقرار می‌کنند. پروسس‌های *host*‌های مختلف هم باید با هم پیام‌هایی را رد و بدل کنند. مثلاً پروسس‌های *client*، درخواست و تماسی را شروع می‌کنند و سمت سرور می‌فرستند. پروسس‌های سرور هم منتظر تماس‌ها برای پاسخگویی به آنها می‌مانند. در معماری *P2P*، چون هر *host* می‌تواند هم سرویس بگیرد هم سرویس ارائه دهد، بنابراین، باید قابلیت‌های هر دو پروسس‌های *client* و پروسس‌های *server* را دارا باشد.

socket

پروسس‌های مبدا و مقصد، از طریق سوکت‌ها اطلاعات را می‌فرستند یا دریافت می‌کنند. درواقع یک سوکت در مبدا است و یک سوکت در مقصد (در مجموع دو سوکت). در واقع مبدا دیتای خود را به یک سوکت می‌فرستد و از طریق سوکت روی شبکه قرار می‌گیرد. مبدا به زیرساخت شبکه در انتقال دیتا اتکا می‌کند. از آن طرف، دیتا از طریق یک سوکتی به مقصد وارد می‌شود. سوکت‌ها مثل درب ورودی یا خروجی می‌مانند.

در وویس استاد گفته شد که بخش‌های لایه‌های پایینی (لایه ۴ تا ۱) سوکت، توسط سیستم عامل کنترل می‌شود. همانطور که گفتیم، پروسس مبدا باید با پروسس مقصد ارتباط بگیرد. پس پروسس‌ها باید شناسه (*identifier*) داشته باشند. پس تنها آدرس *IP* اون *host* کافی نیست چون چندین پروسس روی یک سیستم ران هستند و ما نمی‌دانیم با کدام ارتباط برقرار کنیم.

از آنجایی که هر پروسس با یک پورت در ارتباط است، علاوه بر آدرس *IP* باید شماره پورت آن پردازنده نیز فرستاده شود.

بعضی از شماره پورت‌ها از قبل مشخص شده‌اند. مثلاً برای کار با *email* با پورت ۲۵ سرور و برای *HTTP* با پورت ۸۰ سرور ارتباط می‌گیریم.

موارد قابل ملاحظه در پروتکل

نوع پیام‌های مبادله شده: مثلاً *request* هستند یا *response*.
 نحو *synax* پیام: اینکه پیام از چه بخش‌ها و فیلدهایی تشکیل شده و مثلاً چگونه تشخیص دهیم که فیلد پایان یافته. مثلاً میتوان طول هر فیلد را ذکر کرد.
 معنی *semantic* پیام: یعنی معنی اطلاعات هر فیلد را می‌گوید.
 در پروتکل، قوانینی وجود دارد که می‌گوید پیام‌ها را چگونه و یا در چه زمانی منتقل کنیم.
 بعضی از پروتکل‌ها، *open* هستند (مثل اپلیکیشن‌های *open - source*) و میتوانند توسط افراد مختلف توسعه پیدا کنند. مثل پروتکل‌های *HTTP* و *SMTP*. در مقابل پروتکل‌های خصوصی و اختصاصی مثل اسکایپ که در دسترس عموم نیست.
 اپلیکیشن‌ها برای ارائه بهتر خدمات، سرویس‌های متفاوتی از لایه *transport* میتواند دریافت کند.
 • *data integrity*: اینکه مثلاً بعضی‌ها نباید هیچ کدوم از پکت‌هاشون دراپ بشه یا مثلاً میتونن یه میزان کمی از *packet loss* رو تحمل کنن.
 • *timing*: اینکه مثلاً در یک‌اپ، توقع داریم که *delay* کمی داشته باشیم، مثل بازی‌ها
 • *throughput*: اینکه مثلاً ما یک حداقل *bandwidth* در هر لحظه می‌خواهیم تا کار کنه (مثل‌اپ‌های مولتی‌مدیا و *real - time*) یا اپ‌هایی که با هر *bandwidth* میتونه کارشو انجام بده (مثل‌اپ‌های *elastic*)
 • *security*: اینکه بخواهیم رمزگذاری یا مثل همان بخش اول، یکپارچگی داده داشته باشیم.
 مثلاً یک سری کاربرد‌ها مثل انتقال یا دانلود فایل، یا ایمیل و یا صفحات وب، باید بدون هیچ کم و کاستی با *data integrity* بالا منتقل شوند. ولی می‌توانند *elastic* باشند و به *time* خاصی حساس نیستند و *delay* داشتن اونچنان تاثیر *critical* ندارد. در مقابل، کاربرد‌های *real time* و یا *streaming* و یا بازی‌ها، می‌توانند کمی از *packet loss* را *tolerate* کنند اما باید یک حداقل *bandwidth* در لحظه داشته باشند. همچنین *delay* باید به حداقل و در حد چند میلی‌ثانیه برسد.
 بعضی کاربرد‌ها هم بسته به استفاده می‌تواند مسئله *time* در آن‌ها اهمیت داشته یا نداشته باشد. مثل *text* دادن. در تکست دادن نباید به هیچ وجه *loss* داشته باشیم ولی *bandwidth* می‌تواند به صورت *elastic* باشد.

تیپ پروتکل‌های لایه *transport*

دو نوع سرویس یا تیپ پروتکل در لایه *transport* داریم:

پروتکل‌های TCP

این نوع پروتکل‌ها، یک انتقال اطلاعات *reliable* بین فرستنده و گیرنده برقرار می‌کنند. فرستنده از پر کردن بافر مقصد خودداری می‌کند (*flow control*) و همچنین از به وجود آمدن ازدحام در شبکه و طبعاً *packet loss* جلوگیری می‌کند (*congestion control*). این نوع سرویس به صورت *connection oriented* است و باید یک تماس و *setup* و تنظیمات در اول کار بین پردازنده‌های مبدا و مقصد صورت بگیرد تا کارشان را شروع کنند. این نوع سرویس، زمان‌بندی یا حداقل *bandwidth* یا امنیت را تضمین نمی‌کند.

پروتکل‌های UDP

این پروتکل *reliable* نیست. این پروتکل، *connectionless* است و ارتباطی بین مبدا و مقصد صورت نمی‌گیرد و تنها مبدا، پکت‌ها را می‌فرستد و منتظر ارتباط (*handshake*) نمی‌ماند. هیچ کدام از قابلیت‌های *flow control*، *congestion control*، بحث مربوط به *timing* و حداقل *bandwidth* و امنیت و *connection setup* را تامین نمی‌کند. احتمالاً به همین دلایل، می‌تواند سریع باشد.

مثال

در اسلاید ۱۵ از فصل دوم، مثال‌هایی از پروتکل‌های لایه اپلیکیشن زده شد که از سرویس‌های *UDP* یا *TCP* در لایه *transport* استفاده می‌کنند. مثلاً *FTP* (انتقال فایل) و *SMTP* (ایمیل) و *HTTP 1.1* (برای وب) بر روی *TCP* و پروتکل‌های *SIP* یا *RTP* (تلفن اینترنتی)، *HTTP* یا *DASH* برای *streaming* و برای بازی‌های تعاملی *WOW world of warcraft* یا *FPS = first person shooters* (برای بازی) در لایه انتقال هم از *TCP* و هم از *UDP* می‌توانند استفاده کنند.

TLS

خود *TCP, UDP* امنیت نداشتند و با پروتکل *TLS = transport layer security* تامین کرد که داده را رمزگذاری می‌کرد، یکپارچگی داده و احراز هویت نقطه پایانی را تضمین می‌کند. پیاده‌سازی آن در لایه کاربرد است.

در این پروتکل، متن *clear (plain) text* به سوکت ارسال می‌شود اما به صورت رمزگذاری شده اینترنت را طی می‌کند.

HTTP

http یک پروتکل لایه کاربرد است که برای صفحات و *document* های وب به کار میرود. صفحات وب از *object* هایی تشکیل می‌شوند (و نه بلاک) و این *object* ها میتوانند مثلاً فایل *html*، عکس *JPEG*، فایل صوتی و غیره باشند. درواقع صفحات وب یک فایل *html* پایه دارند که در آن چند تا *reference object* که با هر کدام می‌توان با یک آدرس *url* مراجعه کرد. هر آدرس *url* از دو بخش *host name* (صفحه اصلی سایت) و *path name* (ریز صفحه داخلی آن) تشکیل شده است.

خب برگردیم به *HTTP = hypertext transfer protocol*.

این پروتکل بر اساس معماری *client server* است. *client* از طریق *browser*، در خواستی را به سرور وب یا *http server* ارسال می‌کند. و سرور وب هم تعدادی *object* وب را در پاسخ برایش می‌فرستد. این پروتکل، بر مبنای پروتکل *TCP* در لایه *transport* است و در نتیجه *connection oriented* است. یعنی باید یک *handshake* بین دو طرف صورت بگیرد و سوکت‌ها ساخته شود. در واقع *client* باید تماسی را با پورت ۸۰ سرور که مربوط به *http* می‌شود، برقرار کند.

بعد پیام‌ها بین بروزر (مثلاً فایرفاکس) و وب سرور (مثلاً آپاچی) مبادله می‌شود. بعد باید تماس *TCP connection* قطع شود.

این پروتکل *http*، یک پروتکل *stateless* است؛ یعنی هیچ اطلاعاتی از درخواست های قبلی را در خود ذخیره نمی‌کند، و این درخواست مستقل از درخواست‌های دیگر است.

در مقابل، در پروتکل‌های *stateful*، پیاده‌سازی پیچیده‌تری وجود دارد، چون اولاً باید *state* ها را جایی ذخیره کنیم و ثانیاً ممکن است در صورت قطع یک کدام از *client* یا *server* هایی که با هم تماس دارند، اطلاعات *state* ها از بین برود و با هم ناسازگار شود و دوباره باید بر سر *state* یکسان به توافق برسند.

انواع اتصال‌های HTTP

- اتصال‌های بی دوام *non – persistent*: یعنی هر بار که یک اتصال *tcp* باز شود، حداکثر یک *object* از طریق آن ارسال می‌شود و تماس قطع می‌شود. پس برای چند تا *object* باید چند تا اتصال *tcp* برقرار کرد.
- اتصال‌های بادوام *persistent*: یعنی وقتی اتصال *tcp* برقرار شد، چندین *object* می‌توانند از طریق همان انتقال پیدا کنند و در نهایت تماس بسته می‌شود.

روش HTTP به صورت *non persistent*

مثلاً مراحل پروتکل *non persistent* به این شکل است:

- کاربر تماس را آغاز می‌کند،
- وب سرور که روی پورت ۸۰ منتظر است، تماس را قبول و به کاربر *notify* می‌کند.
- کاربر *request message* را می‌فرستد.
- وب سرور *response message* را می‌فرستد.
- بلافاصله که وب سرور *response* را فرستاد، تماس را قطع می‌کند.
- کاربر *response* را که مثلاً شامل فایل *html* است، دریافت می‌کند. می‌بیند که ۱۰ تا آبجکت عکس *jpeg* در آن است که به صورت *referenced url* است و آن‌ها را باید دوباره با طی مراحل قبلی برای تک‌تک آن‌ها دریافت کند.
- پس در کل مدت زمان آن برابر با $2 \times RTT + \text{file transmission delay}$ که در واقع $RTT = \text{round trip time}$ برابرست با زمان رفت و برگشت یک بسته کوچک. که میشه شامل زمان انتشار رفت و برگشت و صف‌ها و پردازش روترهاست و تقریباً تاخیر ارسال *transmission delay* در آن کم است.
- یک *RTT* برای برقراری اتصال *TCP*. و یک *RTT* هم برای دریافت اولین بیت فایل درخواستی. به علاوه *file transmission delay* تا بیت آخر فایل دریافت شود.
- http 1.0* از نوع بی‌دوام است. که علاوه بر زمان گفته شده، برای هر اتصال *tcp* سربرار در سیستم عامل داریم. نکته مهم این است که بروزر چند تا تماس *tcp* موازی برقرار می‌کند تا *referenced object* های دیگر را به صورت موازی دریافت کند.

روش HTTP به صورت *persistent*

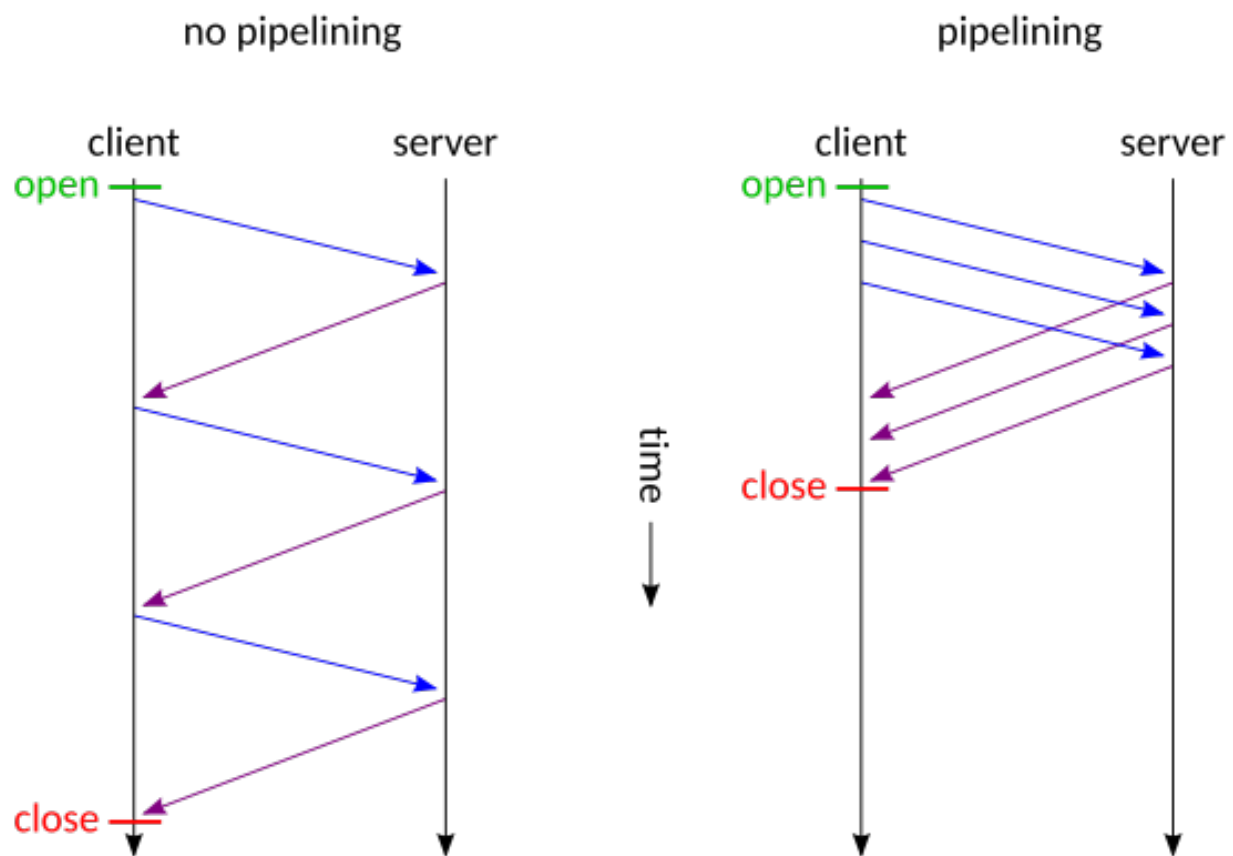
در این روش که از ورژن ۱.۱ در *http* امکان آن فراهم شد، سرور بعد از پاسخ اولیه، تماس را باز نگه میدارد. یعنی بعد از $2 \times RTT + FTD$ ، تماس قطع نمی‌شود و تازه، *client* بلافاصله اون درخواست ۱۰ تا *referenced object* رو برای سرور می‌فرستد و بعد سرور همه آن‌ها را با هم ارسال می‌کند. پس یک تاخیر *RTT* به علاوه تاخیر ارسال ۱۰ *object*، به تاخیر های قبلی اضافه می‌شود.

استفاده از *pipelining* در روش *persistent*

اینکه بتوانیم پشت سر هم چند تا درخواست به سرور ارسال کنیم بدون اینکه منتظر گرفتن جوابشان باشیم. سرور هم پشت سر هم جواب‌ها را ارسال می‌کند. همانطور که در شکل ۱ مشاهده می‌کنید.

HTTP request

درواقع پیام *http request* فرمت خاصی دارد. در خط اول درخواست (*request line*)، *request method* قرار دارد که نوع درخواست ماست. با فاصله یک اسپیس، لینک *url* قرار دارد و به فاصله یک اسپیس ورژن *http* را مینویسیم.



شکل ۱: pipeline

بعد با $\backslash r \backslash n$ به خط بعد می‌رویم.

در خطوط بعدی *header line*ها قرار دارند. در هر خط، یک فیلد به همراه مقدار آن قرار گرفته. مثلاً *host* مقصد چیست یا مثلاً *user agent* کدام مرورگر است. یا اینکه چه فرمت‌هایی از جواب قبول است (مثلاً ۸ - *utf* باشد یا ...) و اینکه تماس به چه مدت برقرار و *keep alive* باشد. و یا نوع ارتباط چگونه باشد مثلاً *keep alive* یا غیره علاوه بر اینکه بعد از هر *header* یک $\backslash r \backslash n$ می‌زنیم تا به خط بعدی برویم، بعد از پایان بخش *header* هم یک خط اضافی، $\backslash r \backslash n$ می‌زنیم (یک خط خالی داریم تا بخش بعدی) و بعد *body* که شامل اطلاعات تکمیلی می‌شود را می‌آوریم.

انواع متدهای *request*

اسم متد ها با حروف بزرگ انگلیسی هستند.

- *POST*: این متد برای نوشتن و پر کردن یک سری فرم است. اطلاعات آن از طریق *entity body* منتقل می‌شود.
- *GET*: برای زمانی که درخواستی (دریافت یک سری آبجکت) از سرور داریم. اطلاعات (متغیرهای) مربوط به آن درخواست را از طریق همان *request line* در بخش *url* مثلاً به صورت *url?a = cat* منتقل می‌کنیم.
- *HEAD*: مثل همان *GET* است اما هنگام پاسخ، *body* را نمی‌فرستد (خالی‌ست). در کارهای دیباگینگ استفاده می‌شود.

- *PUT*: وقتی می‌خواهیم یک آبجکت رو آپلود کنیم یا جایگزینش کنیم، از این متد استفاده می‌کنیم.
- *DELETE*: حذف یک آبجکت از روی سرور.

HTTP response

خط اول معروف به *status line* است. ابتدا ورژن *http* بعد اسپیس بعد *status code* آمده و بعد با یک اسپیس فاصله، عبارت متناظر با آن (*status phrase*) آمده است. *status code* ها اطلاعاتی راجع به موفقیت یا عدم موفقیت *request* و بعضی جزئیات آن را نشان می‌دهند.

مثلاً ۲۰۰ یعنی درخواست موفقیت آمیز بوده اما ۴۰۴ خطای *not found* است. یک جا خوندیم اونی که با ۴ شروع میشه، خطای کاربر و اونی که با ۵ شروع میشه خطای سرور است.

در خط بعدی، *header line*ها قرار می‌گیرند. که در انتهای هر کدام هم $\backslash r \backslash n$ است.

مثلاً *date* یا اینکه *server* چیست یا آخرین باری که تغییر کرده، یا مثلاً *ETag* آن چند است (برای *web caching* کاربرد دارد. یعنی اگر اطلاعات همان صفحه وب در *browser* باشد، دیگر لازم نیست دوباره از سرور بگیرد و همان را نمایش می‌دهد).

یا اینکه طول *content* پاسخ چقدر است. یا اینکه نوع تماس و مدت تماس چقدر است.

مثل *request*، بعد از بخش *header*، یک $\backslash r \backslash n$ میاد و بعد *body* پاسخ میاید که مثلا می تواند *html* یا *text* باشد. (که باز نوع آن از طریق فیلد *connection type* در هدر مشخص می شود)

خودمان می توانیم به طور دستی و بدون استفاده از *browser*، اتصال *TCP* را از طریق *telnet* برقرار کنیم. مثلا:

```
telnet www.google.com 80
```

باز دوباره از طریق برنامه *wireshark* میتوان اطلاعات مربوط به *request* و *response* را رصد کرد.

گزینی به پروتکل های *stateful*

همانطور که می دانید، *http* یک پروتکل *stateless* است یعنی *request* ها مستقل از هم هستند. از درخواست های قبلی نگه نمی دارد.

یعنی برای *multi – step exchange* هیچ *state* نگه نمی دارد.

برای پروتکل های *stateful*، باید *cookies* نگهداری شود که شامل این *state* هاست.

اگر مثلا تغییری روی یک آبجکت در سرور انجام می شود، یا باید به طور کامل انجام شود، یا احتمالا اگر به صورت ناقص انجام شده و ارتباط قطع شده، تغییرات اعمال نشود.

مراجع

[1] “ch2-slide_{1to3}2,” <https://docs.google.com/presentation/>, 2022.